PYTHON PROGRAMMING

NAME : Madineni Dhanush Kumar

REG NO : 1925253188

Course code : CSA0288

ASSIGNMENT 1

TASK 1 - Understand the Problem and Identify the Data (4 Marks)

Bloom's Level: K4 – Analyse

The program calculates milk payments for farmers. Each entry contains farmer ID, litres, fat percentage, SNF percentage and shift. It computes rate per litre, amount for each entry and the ten-day total.

Inputs, outputs, constants and rules

Inputs: Kumar (string, because IDs may contain letters or leading zeroes); litres (fractional numeric value); fat and SNF (fractional percentages); shift (string, such as morning or evening). Outputs: rate per litre, payment amount and ten-day total. Use Decimal for money and preferably for measurements too.

Named constants: BASE_RATE = Decimal('30.00'), FAT_STEP_VALUE = Decimal('0.40'), SNF_STEP_VALUE = Decimal('0.30'), STANDARD_FAT = Decimal('3.5'), STANDARD_SNF = Decimal('8.5'), STEP_SIZE = Decimal('0.1'), MIN_FAT = Decimal('2.0') and MAX_FAT = Decimal('7.0'). Named constants make rate revisions consistent.

Farmer ID and shift are strings; entry count is an integer; litres, fat and SNF may be fractional; money has two decimal places. Rules: litres must be positive, fat must be between 2.0% and 7.0% inclusive, numeric input must be valid, and required text fields cannot be blank. The problem does not state the allowed SNF range, so the co-operative must define it.

Assumptions: the base rate is Rs 30.00 at 3.5% fat and 8.5% SNF; each 0.1 percentage-point fat step changes the rate by Rs 0.40 and each SNF step by Rs 0.30. Ten-day total means all valid entries for one farmer.

Alternative design and complexity

Separate names such as litres1, fat1, snf1 for 300 farmers would create hundreds of variables and make looping, validation and reporting difficult. A list of records stores each entry as one item. Processing n entries takes O(n) time. Storing all entries takes O(n) space; a running total needs O(1) extra space.

TASK 2 - Explore Values and Types in the Interpreter (4 Marks)

Bloom's Level: K4 – Analyse

Interactive interpreter evidence

```
>>> type(10)
<class 'int'>
>>> type(10.0)
<class 'float'>
>>> type("10")
<class 'str'>
>>> type(True)
<class 'bool'>
>>> type([10, 8.5])
<class 'list'>
>>> 7 / 2
3.5
>>> 7 // 2
3
>>> 7 % 2
1
>>> -7 // 2
-4
>>> 0.1 + 0.2 == 0.3
False
```

The / operator performs division; // floors toward negative infinity, so -7 // 2 is -4. The % operator returns the remainder. Binary floating-point cannot represent many decimal fractions exactly, so 0.1 + 0.2 is not exactly 0.3.

Completed Decimal program

```
from decimal import Decimal

payments = ["33.40"] * 1000 + ["0.10"] * 3

def total_float(items):
    total = 0.0
    for amount in items:
        total += float(amount)
    return total

def total_decimal(items):
    total = Decimal("0.00")
    for amount in items:
        total += Decimal(amount)
    return total

print("float :", total_float(payments))
print("Decimal:", total_decimal(payments))
print(0.1 + 0.2 == 0.3,
      Decimal("0.1") + Decimal("0.2") == Decimal("0.3"))
```

Expected output:

```
float : 33400.30000000061
Decimal: 33400.30
False True
```

Decimal(amount) reads the original decimal string exactly. Decimal(float(amount)) first converts to a binary float and then copies its approximation, so the result contains extra digits. Both totals take O(n) time and O(1) extra space beyond the list. Integer paise is another safe option.

TASK 3 - Choose Variables and Name the Constants (3 Marks)

Bloom's Level: K3 – Apply

```
BASE_RATE = Decimal("30.00")
FAT_STEP_VALUE = Decimal("0.40")
SNF_STEP_VALUE = Decimal("0.30")
STANDARD_FAT = Decimal("3.5")
STANDARD_SNF = Decimal("8.5")
STEP_SIZE = Decimal("0.1")
MIN_FAT = Decimal("2.0")
MAX_FAT = Decimal("7.0")
```

Names may contain letters, digits and underscores, but cannot start with a digit; names are case-sensitive and cannot be Python keywords. Use snake_case for variables and UPPER_CASE for constants. Python has no enforced constants, so uppercase is a convention. Python is dynamically typed: value = 10 followed by value = 'ten' is allowed, but later operations must suit the current type. Swap values with a, b = b, a.

Alternative and complexity

If a magic number occurs in k expressions, a revision may require k edits and risks inconsistencies. Named constants generally need one edit. A RATE_CARD dictionary groups related settings. Constant access and dictionary lookup are average O(1), and processing n entries remains O(n).

TASK 4 - Write Correct Expressions (4 Marks)

Bloom's Level: K4 – Analyse

```
fat_steps = round((fat - STANDARD_FAT) / STEP_SIZE)
snf_steps = round((snf - STANDARD_SNF) / STEP_SIZE)
rate = round(BASE_RATE + fat_steps * FAT_STEP_VALUE
             + snf_steps * SNF_STEP_VALUE, 2)
amount = round(litres * rate, 2)
```

Parentheses clarify the intended order. Multiplication and division occur before addition and subtraction. Operators of equal precedence generally associate left to right. Unary minus applies to a value. A chained comparison such as 2.0 <= fat <= 7.0 checks both limits; and/or/not combine conditions.

Surprising expressions

```
>>> (4.2 - 3.5) / 0.1
7.000000000000001
>>> round((4.2 - 3.5) / 0.1)
7
>>> int(-3.7)
-3
>>> import math
>>> math.floor(-3.7)
-4
>>> round(-3.5)
-4
```

The first result is due to floating-point approximation. int() truncates toward zero; math.floor() rounds downward. Python round() uses ties-to-even for halfway cases. For fat 4.2, the step count from 3.5 should be 7. For fat 3.17, the quotient is about -3.3: round() gives -3, int() gives -3 and floor() gives -4. With SNF at 8.5, rates are Rs 32.80 for fat 4.2; for fat 3.17, round/int gives Rs 28.80 while floor gives Rs 28.40. Each method takes O(1) time and O(1) space.

TASK 5 - Read Input and Convert Types (3 Marks)

Bloom's Level: K3 – Apply

input() always returns a string. Use int() for whole-number fields and Decimal() or float() for fractional fields. '8.5' is valid numeric text; 'eight' raises ValueError/InvalidOperation. try/except is a clear way to handle invalid input.

```
from decimal import Decimal, InvalidOperation

text = input("Enter litres: ").strip()
try:
    litres = Decimal(text)
    if litres <= 0:
        print("Litres must be positive")
    else:
        print(f"Litres: {litres:8.2f}")
except InvalidOperation:
    print("Invalid number")
```

The f-string format {amount:8.2f} reserves eight character positions and prints two decimal places. String-method checks can catch empty input but do not reliably validate all decimal formats. Both approaches take O(1) time per entry and O(n) time for n entries.

TASK 6 - Organise the Logic into Functions (4 Marks)

Bloom's Level: K3 – Apply

```
from decimal import Decimal

def rate_per_litre(fat, snf):
    """Return the rate per litre, rounded to two decimals."""
    fat_steps = round((fat - STANDARD_FAT) / STEP_SIZE)
    snf_steps = round((snf - STANDARD_SNF) / STEP_SIZE)
    rate = BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE
    return rate.quantize(Decimal("0.01"))

def payout(litres, fat, snf):
    """Return the payment for one entry."""
    return (litres * rate_per_litre(fat, snf)).quantize(Decimal("0.01"))

def ten_day_total(entries):
    """Return the total payment for a sequence of entries."""
    return sum((payout(*entry) for entry in entries), Decimal("0.00"))
```

A function receives values through parameters and returns a reusable result. print() only displays information. Variables created inside a function are local; module constants can be read by functions but should not be modified there. One long script repeats logic and is harder to test. Lambda is best for short one-expression functions. Function-call overhead is small; total work remains O(n).

TASK 7 - Build a Module (4 Marks)

Bloom's Level: K4 – Analyse. Save this code as milkrate.py:

```
"""Rate card and payment functions for a village milk co-operative."""
from decimal import Decimal

BASE_RATE = Decimal("30.00")
FAT_STEP_VALUE = Decimal("0.40")
SNF_STEP_VALUE = Decimal("0.30")
STANDARD_FAT = Decimal("3.5")
STANDARD_SNF = Decimal("8.5")
STEP_SIZE = Decimal("0.1")
```

```
def rate_per_litre(fat, snf):
    """Return the price per litre, rounded to two decimals."""
    fat_steps = round((fat - STANDARD_FAT) / STEP_SIZE)
    snf_steps = round((snf - STANDARD_SNF) / STEP_SIZE)
    rate = BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE
    return rate.quantize(Decimal("0.01"))

def payout(litres, fat, snf):
    """Return the payment for one entry, rounded to two decimals."""
    return (litres * rate_per_litre(fat, snf)).quantize(Decimal("0.01"))

if __name__ == "__main__":
    examples = [
        (Decimal("10"), Decimal("3.5"), Decimal("8.5")),
        (Decimal("8.5"), Decimal("4.2"), Decimal("8.7")),
        (Decimal("12"), Decimal("3.1"), Decimal("8.3"))
    ]
    for litres, fat, snf in examples:
        rate = rate_per_litre(fat, snf)
        pay = payout(litres, fat, snf)
        print(f"{litres:>5} L fat {fat} snf {snf} rate {rate:6.2f} pay {pay:8.2f}")
```

Run with python milkrate.py. A separate file test_milkrate.py can contain:

```
from decimal import Decimal
from milkrate import payout

print(payout(Decimal("8.5"), Decimal("4.2"), Decimal("8.7")))
```

import milkrate imports the module; from milkrate import payout imports only the named function; import milkrate as mr gives an alias. The if __name__ == '__main__' block runs only when executed directly, so the table is not printed on import. Python searches the script/current folder and locations in sys.path. Imported modules are cached in sys.modules. Copying functions into every script requires repeated edits; a shared module needs one edit. Import has a one-time cost; n payouts take O(n) time.

TASK 8 - Plan the Algorithm Before Coding (4 Marks)

Bloom’s Level: K3 – Apply

Algorithmic problem solving: understand the inputs, outputs and constraints; plan formulas and validation; write pseudocode; implement; test normal, boundary and invalid values; compare expected and actual results.

Pseudocode

```
BEGIN
  SET total = 0.00
  FOR each entry in the ten-day records
    READ farmer_id, litres, fat, snf, shift
    IF entry is invalid THEN
      DISPLAY error and request a valid entry
    ELSE
      fat_steps = ROUND((fat - 3.5) / 0.1)
      snf_steps = ROUND((snf - 8.5) / 0.1)
      rate = ROUND(30.00 + 0.40*fat_steps + 0.30*snf_steps, 2)
      amount = ROUND(litres * rate, 2)
      total = total + amount
      DISPLAY rate and amount
    ENDIF
  ENDFOR
  DISPLAY total
END
```

Flowchart

```
Start -> Read entry -> Validate?
                       | No -> Show error -> Read entry again
                       | Yes -> Calculate steps -> Calculate rate
                               -> Calculate amount -> Add to total
                               -> More entries? -- Yes -> Read entry
                                                -- No -> Show total -> End
```

Hand trace

Entry 1: 10 L, fat 3.5, SNF 8.5 → rate Rs 30.00 → pay Rs 300.00 → total Rs 300.00. Entry 2: 8.5 L, fat 4.2, SNF 8.7 → rate Rs 33.40 → pay Rs 283.90 → total Rs 583.90. Entry 3: 12 L, fat 3.1, SNF 8.3 → rate Rs 27.80 → pay Rs 333.60 → total Rs 917.50.

Storing records uses O(n) space and supports audit, reports and corrections. A running total uses O(1) extra space and is enough if only the final amount is needed. Both take O(n) time.

TASK 9 - Implement and Test the Calculator (4 Marks)

Bloom’s Level: K3 – Apply

```
def validate_entry(litres, fat, snf):
```

```
    """Raise ValueError when an entry breaks the stated rules."""
    if litres <= 0:
        raise ValueError("Litres must be positive")
    if not Decimal("2.0") <= fat <= Decimal("7.0"):
        raise ValueError("Fat must be between 2.0 and 7.0")
    if snf < 0:
        raise ValueError("SNF cannot be negative")
    return True
```

Test case	Input (litres, fat, SNF)	Expected result
Standard	10, 3.5, 8.5	Rate 30.00; pay 300.00
Above standard	8.5, 4.2, 8.7	Rate 33.40; pay 283.90
Below standard	12, 3.1, 8.3	Rate 27.80; pay 333.60
Minimum fat	1, 2.0, 8.5	Valid; rate 24.00; pay 24.00
Maximum fat	1, 7.0, 8.5	Valid; rate 44.00; pay 44.00
Negative litres	-1, 3.5, 8.5	Reject
Fat too low	1, 1.9, 8.5	Reject
Text input	eight	Reject invalid number

Automated checks can use assert, for example: assert rate_per_litre(Decimal('3.5'), Decimal('8.5')) == Decimal('30.00'). Invalid cases should be checked with try/except and the expected ValueError. Automated tests rerun quickly after each edit; manual testing requires retyping cases.

TASK 10 - Compare Expectations with Execution (4 Marks)

Bloom's Level: K4 – Analyse

Float and Decimal totals can differ by tiny amounts due to binary floating-point representation. The difference depends on values and order and may become more noticeable as more values are added. Decimal values constructed from strings preserve decimal amounts.

Measurement program

```
from decimal import Decimal
from time import perf_counter

def measure(n):
    items = ["33.40"] * n
    start = perf_counter()
    float_total = sum(float(x) for x in items)
    float_seconds = perf_counter() - start
    start = perf_counter()
    decimal_total = sum((Decimal(x) for x in items), Decimal("0.00"))
    decimal_seconds = perf_counter() - start
    difference = abs(Decimal(str(float_total)) - decimal_total)
    print(n, float_total, decimal_total, difference, float_seconds, decimal_seconds)

for size in (300, 6000, 150000):
    measure(size)
```

Run the program and record the measured times from your computer; do not invent timings because hardware and Python versions affect them. Both methods take O(n) time. The list uses O(n) space; streaming values can use O(1) extra space.

If each entry is settled separately, round each payment to two decimals before summing. If the bank settles only the combined total, sum exact values and round once. The co-operative's policy determines the correct approach.

A dictionary rate table can cache each (fat, SNF) pair. Lookup is average O(1), as is the original arithmetic formula, but caching can avoid repeated work when calculations are expensive and pairs recur. Space is O(k) for k distinct pairs. Measure with perf_counter() or timeit.

TASK 11 - Recommend and Defend the Final Design (4 Marks)

Bloom's Level: K5 – Evaluate

Use records in a list when audit and reporting are required; separate validation, rate calculation, payout and ten-day total into functions; place rates in one module; create Decimal values from strings; and round rates and payable amounts to two decimal places under a documented settlement policy.

Rate revisions should change named constants or a configuration file rather than multiple expressions. Limitations: the SNF validity range, rounding policy, season dates and bonus interpretation must be confirmed. District-union use would need a database, access control, audit logs, transaction-safe updates and automated tests.

External rate-card file alternative

A JSON or text file can hold rate values and seasonal settings. Load and validate it once at startup. This adds a small startup cost and O(k) space for k settings, but rate revisions can be made without editing and redistributing the program. Validate the file and record its version.

TASK 12 - Re-evaluate and Adapt the Design: Day-4 Design Challenge (8 Marks)

Bloom's Level: K5 – Evaluate

A. Impact analysis (2 Marks)

The analyser now supplies fat and SNF to two decimal places. From April through June, the base rate is Rs 32.00 and each fat step is Rs 0.45. Farmers supplying more than 500 litres in ten days receive a Rs 0.50-per-litre bonus. Rate constants become season-specific. The step size remains 0.1 percentage point unless the union changes it. Two-decimal readings may fall between step boundaries, so the rounding rule must be explicit.

B. Evaluate the original design (2 Marks)

The original functions and module remain useful, but fixed constants are no longer enough. Pass the season's rate card into the calculation function and calculate the bonus after the ten-day litre total is known. Because no seasonal SNF step value is given, this example keeps it at Rs 0.30.

C. Adapted design and working code (2 Marks)

```
from decimal import Decimal, ROUND_HALF_UP

OLD_CARD = {"base": Decimal("30.00"), "fat_step": Decimal("0.40"),
            "snf_step": Decimal("0.30")}
SEASON_CARD = {"base": Decimal("32.00"), "fat_step": Decimal("0.45"),
               "snf_step": Decimal("0.30")}
STEP = Decimal("0.1")
STANDARD_FAT = Decimal("3.5")
STANDARD_SNF = Decimal("8.5")
BONUS_LIMIT = Decimal("500")
BONUS_PER_LITRE = Decimal("0.50")

def steps(value, standard):
    """Round a measurement difference to the nearest 0.1 step."""
    return int(((value - standard) / STEP).quantize(
        Decimal("1"), rounding=ROUND_HALF_UP))

def rate_per_litre(fat, snf, card):
    """Calculate rate using the selected rate card."""
    rate = card["base"] + steps(fat, STANDARD_FAT)*card["fat_step"]
    rate += steps(snf, STANDARD_SNF)*card["snf_step"]
    return rate.quantize(Decimal("0.01"))

def payout(litres, fat, snf, card):
    """Return payment for one entry."""
    return (litres * rate_per_litre(fat, snf, card)).quantize(Decimal("0.01"))

def ten_day_payment(entries, card):
    """Return litres, base payments, bonus and final payment."""
    litres_total = sum((e[0] for e in entries), Decimal("0"))
    payments = sum((payout(*e, card) for e in entries), Decimal("0.00"))
    bonus = litres_total * BONUS_PER_LITRE if litres_total > BONUS_LIMIT else Decimal("0.00")
    bonus = bonus.quantize(Decimal("0.01"))
    return litres_total, payments, bonus, (payments + bonus).quantize(Decimal("0.01"))

if __name__ == "__main__":
    samples = [
        (Decimal("10"), Decimal("4.24"), Decimal("8.67")),
        (Decimal("510"), Decimal("3.50"), Decimal("8.50"))
    ]
    for card in (OLD_CARD, SEASON_CARD):
        print("Card base:", card["base"])
        for entry in samples:
            print(entry, "rate", rate_per_litre(entry[1], entry[2], card),
                  "pay", payout(*entry, card))
        print("10-day total:", ten_day_payment(samples, card))
```

D. Evidence-based defence (2 Marks)

The design keeps calculation logic separate from rate data. Test both rate cards with two-decimal readings and totals of exactly 500, 500.01 and 510 litres. Under the interpretation used here, exactly 500 litres earns no bonus, while 500.01 litres triggers the bonus on all litres. If the union means bonus only on litres above 500, use (litres_total - BONUS_LIMIT) * BONUS_PER_LITRE instead. Confirm this rule before deployment. Run the code and record actual outputs; do not claim unmeasured timings.

SUBMISSION REQUIREMENTS

GitHub repository link (mandatory) containing executed code for every task, completed code programs, input files, output screenshots and README.md. The report should include evidence for Tasks 1–12, diagrams, pseudocode, test tables and complexity analyses. Students may be asked to explain or modify code during a viva.

MARK DISTRIBUTION

Task	**Bloom's Level**	**Marks**
Task 1 – Problem Analysis	K4 – Analyse	4
Task 2 – Values and Types	K4 – Analyse	4
Task 3 – Variables and Constants	K3 – Apply	3
Task 4 – Expressions	K4 – Analyse	4
Task 5 – Input and Type Conversion	K3 – Apply	3
Task 6 – Functions	K3 – Apply	4
Task 7 – Modules	K4 – Analyse	4
Task 8 – Algorithmic Problem Solving	K3 – Apply	4
Task 9 – Implementation and Testing	K3 – Apply	4
Task 10 – Expectation vs Execution	K4 – Analyse	4
Task 11 – Final Recommendation	K5 – Evaluate	4
Subtotal		42
Task 12 – Design Challenge	K5 – Evaluate	8
TOTAL		50